

Università degli Studi di Salerno  
IBM AI Racing League  
in collaborazione con IBM SkillsBuild e Codemotion

---

# Guida autonoma in TORCS

mediante *behavioral cloning*

Diagnosi, controllo manuale e raccolta dati per l'addestramento di un agente di guida

---

**Circuito:** Corkscrew | **Driver:** scr\_server

**Gruppo:** 8

**Componenti:** Santoriello Pasquale  
Fantis Aya  
Crisci Antonio  
Rossomando Andrea  
De Cicco Martina

**Anno accademico:** 2025/2026

# Indice

|          |                                                    |          |
|----------|----------------------------------------------------|----------|
| <b>1</b> | <b>Introduzione e obiettivi</b>                    | <b>2</b> |
| 1.1      | Articolazione del progetto . . . . .               | 2        |
| <b>2</b> | <b>Architettura dell’ambiente</b>                  | <b>2</b> |
| 2.1      | Sensori e comandi . . . . .                        | 2        |
| <b>3</b> | <b>Paradigmi di Apprendimento e Reti Neurali</b>   | <b>3</b> |
| <b>4</b> | <b>Dataset</b>                                     | <b>3</b> |
| 4.1      | Raccolta dati e Dataset . . . . .                  | 3        |
| <b>5</b> | <b>Addestramento</b>                               | <b>4</b> |
| 5.1      | Pre-processing . . . . .                           | 4        |
| 5.2      | L’Architettura Multi-Task: TorcsNet . . . . .      | 4        |
| 5.3      | Vincoli Fisici e Funzioni di Attivazione . . . . . | 5        |
| 5.4      | Ottimizzazione e Steer-Weighted Loss . . . . .     | 5        |
| 5.5      | Impostazioni di Addestramento . . . . .            | 5        |
| <b>6</b> | <b>Guida Autonoma</b>                              | <b>6</b> |
| 6.1      | Ciclo di Guida . . . . .                           | 6        |
| 6.2      | Risultati Finali . . . . .                         | 6        |
| <b>7</b> | <b>Conclusioni</b>                                 | <b>6</b> |

## 1 Introduzione e obiettivi

Il presente documento descrive il lavoro svolto nell'ambito della competizione **IBM AI Racing League**, organizzata dall'Università degli Studi di Salerno in collaborazione con IBM SkillsBuild e Codemotion. L'obiettivo della competizione è lo sviluppo di un agente di intelligenza artificiale capace di guidare in modo autonomo un'automobile all'interno del simulatore **TORCS** (*The Open Racing Car Simulator*), nella sua versione controllabile via Python, percorrendo il circuito **Corkscrew** nel minor tempo possibile, senza uscire di pista e senza perdere aderenza.

Il veicolo è pilotato attraverso il driver `scr_server`, che espone i sensori del veicolo e accetta i comandi di controllo tramite una connessione di rete (socket UDP). La strategia adottata dal gruppo è il **behavioral cloning**, un approccio di apprendimento supervisionato in cui un modello impara a guidare osservando i dati di guida prodotti da un pilota umano, implementato tramite una **rete neurale**.

### 1.1 Articolazione del progetto

Il progetto si articola in tre fasi logiche:

1. **Raccolta dati.** Un operatore umano guida il veicolo manualmente registrando, a ogni istante, lo stato dei sensori e l'azione di controllo corrispondente.
2. **Addestramento.** Un modello di apprendimento viene addestrato sui dati raccolti, affinché impari ad associare ogni stato sensoriale alla relativa azione.
3. **Guida autonoma.** Il modello addestrato sostituisce il pilota umano e controlla il veicolo in autonomia.

## 2 Architettura dell'ambiente

L'ambiente di lavoro è costituito da TORCS in esecuzione su sistema Windows e da un insieme di moduli Python che comunicano con il simulatore. I componenti rilevanti per il progetto sono i seguenti.

- `manual_control_ps4_1.py` — modulo di guida manuale tramite il controller PS4, che traduce i tasti in comandi di sterzo, accelerazione e freno. Implementata usando la libreria `pygame`, che è progettata per lo sviluppo di giochi. Registra ogni nostra azione in tempo reale per creare i file CSV necessari per l'addestramento.
- `training_modello.py` — è il file che acquisisce i nostri giri realizzati con la guida manuale (file CSV), e li trasforma nei pesi della rete neurale in grado di guidare in modo autonomo.
- `guida_bc_nc_1.py` — script che abilita la guida autonoma del veicolo all'interno del simulatore, tramite il modello Behavioral Cloning. In questo modo permette di replicare la traiettoria e lo stile di guida appresi dal dataset.

### 2.1 Sensori e comandi

Lo stato osservabile del veicolo, utilizzato come input del modello, comprende le grandezze riportate nella Tabella 1. I comandi di controllo (output) sono lo sterzo, l'accelerazione, il freno. Mentre le feature di ingresso sono: `speedX`, `angle`, `trackPos`, `rmp`, `damage`, `track[0...18]`.

Tabella 1: Principali grandezze sensoriali e comandi del veicolo.

| Grandezza     | Tipo    | Significato                                                           |
|---------------|---------|-----------------------------------------------------------------------|
| speedX        | sensore | Velocità longitudinale (km/h)                                         |
| angle         | sensore | Angolo del veicolo rispetto all'asse pista                            |
| trackPos      | sensore | Posizione laterale rispetto al centro pista                           |
| track[0...18] | sensore | 19 sensori telemetrici che misurano la distanza dal bordo della pista |
| rpm           | sensore | Numero di giri al minuto del motore dell'auto                         |
| damage        | sensore | Danno attuale dell'auto (più alto è il valore, maggiore è il danno)   |
| steer         | comando | Sterzo, in $[-1, 1]$                                                  |
| accel         | comando | Accelerazione, in $[0, 1]$                                            |
| brake         | comando | Freno, in $[0, 1]$                                                    |

### 3 Paradigmi di Apprendimento e Reti Neurali

Per il nostro progetto abbiamo scelto come approccio il **Behavioral Cloning** perchè è semplice, intuitivo e veloce. Non abbiamo usato il Reinforcement Learning in quanto richiede simulazioni infinite, che faccia tanti tentativi casuali prima di capire cosa sia giustamente fare. Una volta scelto il BC, serviva un modello per apprendere le relazioni non lineari tra i sensori del veicolo e i comandi di guida.

La **Rete Neurale** è stata scelta perchè, anche se richiede molto tempo e potenza di calcolo, racchiude tutta la conoscenza nei suoi 'pesi' (matrici matematiche). Quando l'auto è nella pista, l'algoritmo deve fare solo una serie di moltiplicazioni matematiche veloci. La risposta ci mette pochissimo tempo, indipendentemente da quanto fosse enorme il dataset iniziale. Più dati forniamo e più diventa precisa e robusta.

Proprio per questo lo abbiamo preferito rispetto al k-NN che invece più raccogliamo i dati e più diventava lento e pesante da mantenere.

## 4 Dataset

### 4.1 Raccolta dati e Dataset

Il primo step del progetto è stato quello di raccogliere i dati necessari per l'addestramento del modello di Behavioral Cloning.

Tramite lo script *manual\_control\_ps4\_1.py* e grazie all'uso del controller PS4, abbiamo registrato i nostri giri di guida manuale nel simulatore. Nel corso dello svolgimento del progetto abbiamo sempre di più ampliato il nostro dataset con l'aggiunta di nuovi giri, cercando di registrare più situazioni diverse. All'inizio ci siamo focalizzati sulla realizzazione di traiettorie ideali, con la macchina sempre al centro della pista e puntando ad un ottimo tempo.

Successivamente, ci siamo resi conto che la rete aveva bisogno anche di imparare situazione e traiettorie diverse: per questo abbiamo aggiunto volutamente giri imperfetti, portando il veicolo sui bordi della pista per dimostrare all'agente come correggere l'angolo di sterzo e di rientrare al centro del percorso.

In totale sono stati completati 55 giri, realizzando un dataset di 48.453 campioni.

Il dataset raccolto tramite *manual\_control\_ps4\_1.py* riflette il paradigma del Behavioral Cloning. Esso mappa un vettore di **Feature**: speedX, angle, trackPos, angle, rpm, damage, track[0...18]; e un vettore di **Target** che rappresentano le azione di guida che abbiamo preso usando il controller PS4, che sono:

- **steer**: il comando dello sterzo gestito tramite la levetta analogica sinistra del controller. Il valore è limitato matematicamente all'intervallo  $[-1, 1]$ , dove i due estremi rappresentano la sterzata massima a sinistra e a destra.
- **accel**: il pedale dell'acceleratore, catturato dal grilletto progressivo R2, che è normalizzato nel range  $[0, 1]$ .
- **brake**: il pedale del freno, catturato dal grilletto progressivo L2, anch'esso normalizzato nel range  $[0, 1]$ .

Nella Tabella 2, viene riportato un estratto dei primi campioni catturati dal sistema di telemetria durante la fase di partenza da fermo del veicolo. Dall'analisi di questi record si possono evincere delle proprietà fondamentali della struttura dati:

- **Frequenza di campionamento**: L'evoluzione temporale della colonna `time` evidenzia un campionamento costante con un passo di circa 0.1 s (10 Hz). Tale frequenza garantisce la corretta cattura della dinamica del veicolo senza sovraccaricare la memoria durante la fase di addestramento.
- **Filtrazione cinematica degli input**: Osservando la colonna `accel`, si nota come il comando non si comporti come un gradino istantaneo da 0.0 a 1.0. Il valore cresce in modo fluido e progressivo ( $0.0 \rightarrow 0.496 \rightarrow 0.848 \rightarrow 0.933$ ), dimostrando l'efficacia del filtro di *smoothing* implementato per evitare il pattinamento delle ruote motrici in accelerazione.

Il freno (`brake`), infine, si attesta stabilmente sul valore nullo (0.0), coerentemente con la necessità di massimizzare lo spunto iniziale del veicolo.

Tabella 2: Estratto completo della telemetria iniziale comprensivo di target e feature cinematiche.

| time  | steer | accel | brake | gear | speedX | trackPos | angle | rpm  | track_0 | ... | track_18 |
|-------|-------|-------|-------|------|--------|----------|-------|------|---------|-----|----------|
| 0.119 | 0.0   | 0.000 | 0.0   | 1    | -0.009 | 0.333    | 0.008 | 5236 | 5.703   | ... | 11.222   |
| 0.219 | 0.0   | 0.000 | 0.0   | 1    | -0.009 | 0.333    | 0.008 | 5236 | 5.703   | ... | 11.222   |
| 0.339 | 0.0   | 0.496 | 0.0   | 1    | -0.009 | 0.333    | 0.008 | 5417 | 5.703   | ... | 11.222   |
| 0.457 | 0.0   | 0.848 | 0.0   | 1    | -0.009 | 0.333    | 0.008 | 6147 | 5.703   | ... | 11.222   |
| 0.561 | 0.0   | 0.933 | 0.0   | 1    | -0.009 | 0.333    | 0.008 | 7008 | 5.703   | ... | 11.222   |

## 5 Addestramento

### 5.1 Pre-processing

Prima dell'addestramento i dati sono stati esposti a una fase di pulizia, rimuovendo gli step iniziali con velocità nulla, quelli con posizione fuori pista e quelli con il danno elevato.

Le feature di input sono state normalizzate tramite *StandardScaler*, sottraendo la media e dividendo per la deviazione standard, operazione necessaria per garantire la convergenza della rete neurale. Il dataset è suddiviso poi in training set (80%) e test set (20%).

### 5.2 L'Architettura Multi-Task: TorcsNet

La complessità del controllo veicolare ha richiesto l'implementazione di un'architettura di tipo Multi-Task Learning, denominata *TorcsNet*. La topologia della rete, sviluppata tramite il framework *PyTorch*, è strutturata in due blocchi sequenziali:

- **Shared Layers**: Una spina dorsale comune composta da tre layer densamente connessi (rispettivamente da 128, 64 e 32 neuroni), intervallati da funzioni di attivazione *ReLU* (Rectified Linear Unit). Questo blocco ha il compito di estrarre le *feature* spaziali e cinematiche globali dall'input a 22 dimensioni.

- **Branching Heads:** L'ultimo layer condiviso si dirama in tre nodi di output indipendenti (*steer\_head*, *accel\_head*, *brake\_head*), ciascuno dedicato alla predizione di uno specifico comando meccanico.

### 5.3 Vincoli Fisici e Funzioni di Attivazione

Per fare in modo che l'intelligenza artificiale (*TorcsNet*) guidi come una macchina vera e non compia azioni impossibili per il simulatore, è stato necessario porre dei limiti matematici ai suoi comandi.

- **Controllo Sterzo:** Per il controllo laterale è stata usata la funzione (*Tanh*), che limita il valore di uscita nell'intervallo  $[-1, 1]$ . Questo rispecchia il comportamento del volante: -1 corrisponde alla massima sterzata a verso destra, +1 verso sinistra.
- **Controllo Pedali:** Per i controlli longitudinali è stata implementata *Sigmoid*, che limita i valori nell'intervallo  $[0, 1]$ . La Sigmoid garantisce quindi che la rete non produca mai comandi fisicamente impossibili.

### 5.4 Ottimizzazione e Steer-Weighted Loss

Durante l'addestramento si è scelto di non trattare tutti e tre i comandi di output allo stesso modo. Intuitivamente, commettere un errore nello sterzo è molto più grave che commettere un errore sull'acceleratore: una traiettoria sbagliata porta la macchina fuori pista immediatamente, mentre una leggera imprecisione sul gas influisce solo sulla velocità. Per riflettere questa differenza, invece di usare una funzione di errore standard uguale per tutti gli output, è stata definita una funzione di perdita personalizzata che assegna un peso diverso a ciascun comando:

$$\mathcal{L}_{total} = w_1 \mathcal{L}_{steer} + w_2 \mathcal{L}_{accel} + w_3 \mathcal{L}_{brake} \quad (1)$$

I pesi sono stati scelti sperimentalmente e impostati a  $w_1 = 5.0$  per lo sterzo,  $w_2 = 1.0$  per l'acceleratore e  $w_3 = 2.0$  per il freno. In questo modo, durante l'aggiornamento dei pesi della rete, l'ottimizzatore viene "forzato" a dare maggiore priorità alla correttezza della direzione rispetto agli altri comandi, migliorando la stabilità della traiettoria in curva.

### 5.5 Impostazioni di Addestramento

L'addestramento della rete è stato eseguito sfruttando la GPU. Per aggiornare i pesi della rete ad ogni iterazione è stato scelto l'ottimizzatore *Adam*, con un learning rate di 0.001. Adam è stato preferito ad altri ottimizzatori perché adatta automaticamente il passo di aggiornamento per ciascun peso, risultando più stabile e veloce da configurare.

Per evitare l'overfitting, ovvero il rischio che la rete memorizzi i dati di training senza saper generalizzare su situazioni nuove, è stata applicata la tecnica del Dropout con probabilità  $p = 0.1$  negli strati nascosti condivisi. Ad ogni iterazione, il 10% delle connessioni neurali viene disattivato casualmente, costringendo la rete a non dipendere da percorsi fissi e a sviluppare rappresentazioni più robuste.

- **modello\_torcs\_pytorch.pth:** contiene i pesi acquisiti dalla rete neurale durante l'addestramento. Viene salvato tramite *torch.save()* di PyTorch.
- **scaler\_torcs.pkl:** contiene lo *StandardScaler*, cioè i valori di media e deviazione standard calcolati sul training set. È importante salvarlo insieme al modello perché a runtime il driver deve normalizzare i dati dei sensori nello stesso modo in cui sono stati normalizzati durante l'addestramento.

## 6 Guida Autonoma

Una volta completato l'addestramento, il modello viene usato da *guida\_bc\_nc\_1.py* che consente di guidare la macchina in modo autonomo all'interno del simulatore.

All'avvio, lo script carica il modello addestrato *modello\_torcs\_pytorch.pth* e lo scaler *scaler\_torcs.pkl*, e dopo stabilisce la connessione con Torcs grazie alla libreria *SnakeOil*.

### 6.1 Ciclo di Guida

Ad ogni fase della simulazione, lo script esegue sempre la stessa sequenza di operazioni:

- **Lettura dei sensori** – Vengono letti i 22 valori di input: velocità (**speedX**), posizione laterale (**trackPos**), angolo rispetto alla pista (**angle**) e i 19 valori del sensore **track**.
- **Normalizzazione** – I valori vengono normalizzati tramite lo scaler, applicando la stessa trasformazione usata durante l'addestramento.
- **Inferenza** – Il vettore normalizzato viene passato alla rete neurale **TorcsNet**, che produce in output i tre comandi: sterzo, acceleratore e freno.
- **Invio comandi** – I valori predetti vengono inviati al simulatore, con un piccolo moltiplicatore applicato allo sterzo per aumentarne la reattività.

Il modello non è addestrato a gestire le marce, in quanto seleziona la marcia ottimale in base alla velocità corrente del veicolo.

Il modello deve aver imparato dai dati di guida manuale tutto ciò che serve per guidare autonomamente, senza controlli o regole aggiuntive imposte dall'esterno.

### 6.2 Risultati Finali

Il nostro obiettivo principale è stato quello di garantire la stabilità dinamica dell'auto ed evitare danni. L'approccio con il Behavioral Cloning infatti si è rivelato ottimo in tal senso, permettendo alla rete neurale di imitare con successo le traiettorie estratte dal dataset e di essere in grado di completare un giro in autonomia.

In seguito, dopo che ci siamo assicurati che il veicolo fosse in grado di mantenere una stabilità all'interno dei limiti della pista, la nostra attenzione si è spostata nel cercare di perfezionare il tempo.

Nel dataset iniziale erano presenti giri ottimi con un tempo pari a 1 minuto e 13 sec, a cui abbiamo dovuto aggiungere altri giri più lenti e che ci permettessero di mappare situazioni di recupero e traiettorie diverse.

Per questo, facendo una media di questi comportamenti, il modello ha realizzato una guida fluida ma con un tempo pari a 1:45.

## 7 Conclusioni

In sintesi, i risultati del progetto ci hanno dimostrato che l'auto è in grado di guidare in modo autonomo, grazie al modello che ha appreso come coordinare lo sterzo, acceleratore e freno senza alcun tipo di 'intervento esterno', completando il percorso in modo fluido e sicuro con un tempo pari 1:45.